
A Comparative Study On Automating Code Review Using Pre-Trained LLMs and Vanilla Transformer-Based Model

Yeabsira Bekele

opt534@mail.usask.ca

Abstract

Code review is a crucial step in the software development cycle. It involves a comprehensive evaluation of source code by peers to enhance its quality and address defects before integration into the main code base. This process involves code submission, reviewer selection, examination, feedback, revisions, and final approval. It facilitates error detection, knowledge transfer among team members, and adherence to coding standards, contributing to a more robust and consistent code base. Despite its benefits, the code review process comes at a cost. It incurs additional expenses and demands significant developer effort and time. To alleviate these challenges and enhance efficiency in the software development cycle, empirical studies explore the application of transformer-based models to automate the code review process. While these studies hold promise for fully automating code reviews, a comparative analysis of using pre-trained versus vanilla transformer-based models remains a subject for further investigation. This study assesses the accuracy of code implementations generated by a fine-tuned Language Models (LM) Code-Llama, Llama-3, CodeT5+, CodeGemma, and the benchmark OpenNMT-tf transformer-based model (1-encoder by 2-decoder), which is trained from scratch. It targets the code review process from the contributor's side, offering the developer a quick and automated improved code implementation of the code to be submitted for review. The transformer-based models take in the code submitted by the developer and generate an improved implementation of the code provided. The result shows that the seven billion-parameter Code-Llama outperforms the benchmark model with perfect code prediction by 5.82% (three times the performance).

1 INTRODUCTION

Code review is an integral process in software development, involving a systematic examination of source code by peers to enhance overall code quality and identify and fix defects before integration into the main code base. The process encompasses code submission, reviewer selection, examination, feedback, revisions, and final approval. Code review enables error detection, knowledge transfer among team members, and the enforcement of coding standards, contributing to a higher-quality and more consistent code base. However, it also comes with potential drawbacks. Code reviews add additional expenses to the standard development costs due to allocating one or more reviewers responsible for verifying the correctness, quality, and soundness of newly developed code. Developers spend a lot of effort reviewing code. This represents a cost in terms of time and pushes developers to switch contexts from their tasks.

The emergence of Large Language Models (LLMs) has had a transformative impact on software engineering. To ease the workload the software developer community endures during the code review

process and the overall inefficiencies incurred in the software development cycle, empirical studies have been conducted to automate the code review process. These studies explore using pre-trained and vanilla transformer-based models to automate the code review process. While these studies are promising and can soon mature to the point where the code review process can be fully automated, a comparative study of whether to use a pre-trained model and fine-tune it or build a model from scratch (training from scratch) provides a strong motivation for future studies. Although it is tempting to assume fine-tuned models far outperform models trained from scratch, a thorough empirical analysis is required to get a definitive answer. This study compares the accuracy of the revised code implementation generated by fine-tuned LLM, Code-Llama, and the benchmark OpenNMT-tf [10], [11] transformer-based model (1-encoder by 2-decoder) trained from scratch. Instead of performing comparative analysis on studies that have attempted to automate the code review process using vanilla transformer-based models or pre-trained models, this study considers maintaining a similar design environment to ensure the result is meaningful and less prone to bias.

The code review process is generally perceived from two perspectives: the contributor's (developer submitting code for review) and the reviewer's. From the contributor's point of view, the code review process involves writing and submitting a clear code for reviews, being open to feedback and suggestions from the reviewers) and understanding and addressing the review comments by providing improved code implementation. From the reviewer's point of view, the code review process largely revolves around understanding the context and purpose of the code changes, reviewing the code for correctness, efficiency, readability, and adherence to coding standards, identifying potential bugs, edge cases, or areas for improvement and finally providing constructive feedback and suggestions for improvement. This comparative study targets the code review process from the contributor's perspective and compares models that offer the contributor a quick and automated implementation of the code to be reviewed. The transformer-based models under comparison take in the code submitted by the contributor and generate an improved implementation of the code provided. To carry on this study, the pre-trained LM Code-Llama 7B [6], Llama-3 8B [28], CodeT5+ 220m, CodeT5+ 770m [24], CodeGemma 7B [29] are fine-tuned on the same dataset on which the benchmark model has been trained. The results from these models are then evaluated and compared to better understand whether using pre-trained models to automate the code review process is better.

2 RELATED WORKS

With the advent of transformer-based models there has been a growing interest in automating various downstream tasks such as code generation and completion [25], program testing [26] and repair [27], and the code review process [1, 2, 3, 22, 23]. Following up on the automation efforts of the code review process to prevent software developers from expending too many resources, such as prolonged effort and time, empirical studies explore the use of LLMs to automate the code review process. Among the existing work lies the study by Palvannan et al. [1], which provides a Suggestion Bot to automatically review the code base using GitHub's suggested changes functionality to resolve the issue and offer the developer meaningful feedback. This study aims to overcome the challenges developers face from the poor interactions provided by existing software bots due to incomprehensible feedback and workflow disruption.

The study by Tufano et al. [2] titled "Towards Automating Code Review Activities" explores using Neural Machine Translation (NMT) to partially automate code review activities. It collects code review data, preprocesses it to extract methods and link reviewer comments, and filters out noisy comments. Two datasets are created: Dp (paired data containing submitted code and reviewed code) for training a 1-encoder model to recommend changes to developers, and Dt (triplet data that includes the reviewer comment in addition to the Dp) for training a 2-encoder model to recommend changes to reviewers based on the submitted code and developer's response. Both models are evaluated using perfect prediction, BLEU-4 [20], and Levenshtein distance [21] scores. The 1-encoder model shows that it can automatically implement certain types of code changes that could be suggested by reviewers. The 2-encoder model is evaluated on its ability to predict the developer's response, taking the initial code submitted by the developer and the comment provided by the reviewer into account, with promising results. The study demonstrates the potential of NMT techniques to assist in automating parts of the code review process. A subsequent study by Tufano et al. [23] explores applying pre-trained models, specifically the Text-to-Text Transfer Transformer (T5), to automate various aspects of the code review process. This study aims to enhance the efficiency and effectiveness

of code reviews by leveraging the capabilities of pre-trained models to perform tasks traditionally done by human reviewers. It also aims to address the growing need for automated tools in software development that can assist in code review by predicting code changes, implementing recommended changes, and generating natural language comments for code review. It constructs datasets for pre-training and fine-tuning phases from Stack Overflow and CodeSearchNet, ensuring the model learns general knowledge about programming and technical language and task-specific knowledge. The T5 model is first pre-trained on a dataset focusing on Java and technical English. It is then fine-tuned on specific code review tasks. The paper outlines a detailed evaluation framework that assesses the model’s ability to predict and implement code changes and generate review comments.

The LLaMa-Reviewer [3] aims to automate three core tasks integral to the code review process: review necessity prediction, code review comment generation, and code refinement. These tasks correspond to stages in a typical code review process and involve inputs and outputs in programming language (PL) for code snippets, natural language (NL) for code comments, and binary labels (L) for decisions on the requirement for further review. The study uses the 6.7B parameters variant of LLaMA as the base model. It employs the CodeReviewer (multi-language) and the Tufano (Java-specific) datasets. The evaluation criteria include precision, recall, and F1-score for review necessity prediction and BLEU-4 score for code review comment generation and refinement tasks. The paper explored the impact of instruction tuning and various parameter-efficient fine-tuning (PEFT) methods on the model’s performance. LLaMAReviewer achieves competitive performance with less than 1% of the trainable parameters and significantly reduces storage space requirements by choosing an appropriate LoRA rank. The study by Sghaier et al. [22] on improving the Learning of Code Review Successive Tasks with Cross-Task Knowledge Distillation explores the application of cross-task knowledge distillation in the context of code review tasks. It introduces a method where knowledge is transferred from a model trained on one task to another model trained on a different but related task. This approach aims to enhance the performance of models in automating code reviews by using a teacher-student model framework, where the student model learns not only to predict accurate outputs but also to assist the teacher model in refining its predictions. The study demonstrates that this method can improve accuracy and efficiency in code review processes.

3 Methodology

This study first prepares a similar environment for comparison by preparing a dataset on which the models under comparison will be trained. The next step entails carefully selecting pre-trained LLMs to be trained on the dataset and then compared to the benchmark vanilla transformer-based model (OpenNMT-tf) to assess which is more suitable for automating the code review process. The selection criteria for each model are described in subsection B: Model Choice.

After model training, the improved code implementation on the testing data is evaluated based on perfect prediction, BLEU-4, and Levenshtein Distance scores. Perfect prediction is the main evaluation metric considered for the benchmark study, so it will take priority in evaluating the models under comparison. However, BLEU-4 and Levenshtein distance scores have also been considered to provide more context on the model performance. The perfect prediction score is either one or zero. The model is said to have a perfect prediction score of 1 when the code generated by the model is the same as the actual improved code implemented by humans on the testing dataset. If the model’s prediction is anything other than that, it will be assigned a perfect prediction score of zero for every data point it fails to predict accurately.

BLEU (Bilingual Evaluation Understudy) [20] is a metric used to assess the quality of machine-generated text (model’s code prediction) against reference text (human-improved implementation of the code). BLEU-4 specifically measures the overlap of 4-grams in the model’s output, which also appears in the reference code. BLEU-4 ranges from 0 to 1, with higher scores indicating more similarity to the reference code. A score of 1 means the model’s output is the same as the reference code. As the value for the BLEU-4 decreases, it signals that the code generated by the model is less similar to the reference code. Levenshtein distance [21] is often used to evaluate the quality of text generation, paraphrase generation, and text summarization tasks by comparing the machine output to reference texts. It measures the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one string into another. It is a string metric that quantifies the similarity between two sequences of code tokens. For ease of interpretation, the Levenshtein distance can be converted to a similarity ratio by dividing the minimum edit distance by the number of tokens

in the longest sequence among the predicted and the reference code. Doing so also maintains the metric design from the benchmark study. After normalization, the values range from zero to one. Having a zero value means the prediction and reference code are the same, as the minimum number of edits required to match them is zero. The similarity between the prediction and reference code decreases as the value increases.

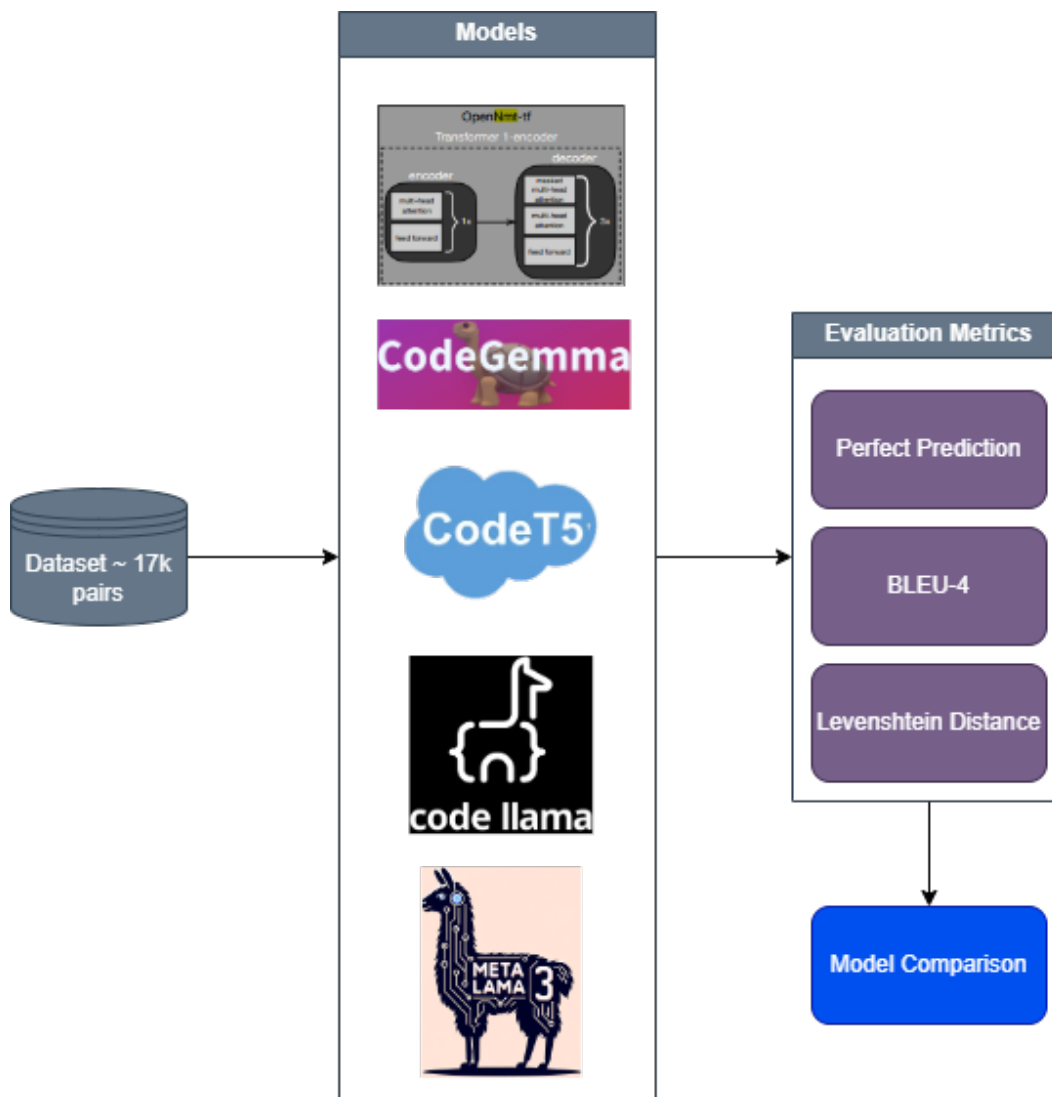


Figure 1: Approach overview

Figure 1 illustrates the approach for the comparative study. Subsections A and B outline the dataset preparation and model choice, respectively.

(A) Dataset

Maintaining a particular design environment allows for the comparison of different models to be performed with minimal bias. To set the right stage for this comparative study, the design environment implemented by Tufano et al. [1] is used as a benchmark. The design environment includes the data used and the methodology applied. The difference comes when training the different models under comparison. The raw dataset mined from GitHub [12] (2,566 Java repositories having at least 50 pull requests) and Gerrit [13] (6,388 Java projects) by Tufano et al. [1] is available for replication purposes.

From the data mined from GitHub and Gerrit, the code submitted by a contributor for review is parsed to extract the list of methods it contains. Only small and well-defined

code units represented by methods were chosen. In doing so, all submitted methods (m_s) are first identified. Then, the reviewer’s comments on each m_s are collected by exploiting the information available in both GitHub and Gerrit, linking a reviewer’s comment to a specific source code line. Each comment is referred to as r_{nl} (i.e., a natural language recommendation made by a reviewer). The review comments are then filtered to discard comments that are unlikely to make recommendations that result in code changes (e.g., "thank you," "well done," etc.).

If the contributor decides to address (some of) the received r_{nl} , this will result in a revised version of m_s addressing the received comments. This revised version of m_s is referred to as m_r . Both m_s and m_r are abstracted to reduce the vocabulary size and make them more suitable for Deep Learning (DL) [14], [15], [16]. To increase the likelihood of m_r being the actual improved implementation of m_s based on the review comment, r_{nl} , only m_s that received a single comment in a review round is considered. Doing this ensures that if a revised version of m_s is submitted, it can be assumed that it implements a single comment made by a reviewer.

Such a process results in a dataset of triplets – code submitted for review, review comment, and revised version of code submitted: $\{(m_s, r_{nl}) \rightarrow m_r\}$. Since this study is concerned with the code review process from the contributor’s point of view, the reviewer’s comment is removed from the triplet dataset to create a dataset consisting of code submitted for review and the revised version of the code submitted. A dataset constructed in such a way can be used to train transformer-based models by feeding the Java method to generate an improved implementation of the code: $\{m_s \rightarrow m_r\}$

Figure 2 below illustrates the approach designed by the benchmark study. Even though the reviewer’s comment is not part of the dataset used for this study, it is included in the initial data pre-processing explanation for a better context of the underlying process.

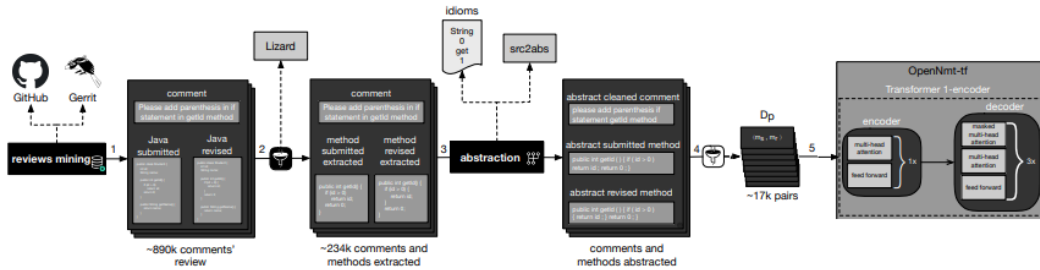


Figure 2: Benchmark Approach Overview

(i) Methods of Extraction and Abstraction

The Java files involved in the review process (those submitted for review and implementing the code review comments) are parsed using the Lizard [17] Python library to extract the methods from all the files. For each mined review round, the list of Java methods submitted for review, the reviewers’ comments, and the revised list of methods resubmitted by the author to address (some of) the received comments are present.

Then, the abstraction process described in Tufano et al. [14] is adopted to obtain a vocabulary-limited yet expressive representation of the source code. Recent work on generating assert statements using DL [16] showed that the performance of sequence-to-sequence models on code is substantially better when the code is abstracted with the procedure presented in [14] and implemented in the src2abs tool [18]. Triplets for which a parsing error occurs during the abstraction process on the m_s or on the m_r methods are removed from the dataset. Figure 3 illustrates an example of the abstraction procedure performed. The top part represents the raw source code. src2abs uses a Java lexer and a parser to represent each method as a stream of tokens, in which Java keywords and punctuation symbols are preserved, and the role of each identifier

(e.g., whether it represents a variable, method, etc.) as well as the type of a literal, is discerned.

raw source code

```
public PageProperties getProperties() {
    if (hasProperties()) {
        return properties;
    } else {
        return null;
    }
}
```

abstracted code

```
public TYPE_1 METHOD_1 ( ) { if ( METHOD_2 ( ) )
{ return properties ; } else { return null ; } }
```

Figure 3: Example of abstraction

IDs are assigned to identifiers and literals by considering their position in the method to undergo the abstraction process. The first variable name found will be assigned the ID of "VAR 1"; likewise, the second variable name will receive the ID of "VAR 2". This process continues for all identifiers and the literals (e.g., STRING X, INT X, FLOAT X). Since some identifiers and literals appear very often in the code (e.g., variables *i*, *j*, literals 0, 1, method names such as "size"), those are treated as "idioms" and are not abstracted. The list of idioms is constructed by looking for the 300 most frequent identifiers and literals in the extracted methods. The bottom part of Figure 2 shows the abstracted version of the source code. One thing to note is that code comments are removed during the abstraction process. src2abs is particularly well suited for the abstraction in this context as it implements a "pair abstraction mode," in which a pair of methods can be provided (in this case, m_s and m_r , and the same literals/identifiers in the two methods will be abstracted using the same IDs. As the output of the abstraction process, src2abs also provides an abstraction map *M* linking the abstracted token to the raw token (e.g., mapping VAR 1 to sum). This allows the abstracted code to be reverted to the raw source code [14].

(ii) **Linking and Abstracting Reviewer Comments**

For the data mined from Gerrit and GitHub, each collected reviewer comment is associated with the specific set of code lines it refers to. Based on this information, each comment can be linked to the method (if any) it refers to. To do so, given the start (l_s) and end (l_e) lines a given comment refers to, the comment can be linked to a method (m_i) if both l_s and l_e fall within m_i 's body, signature, or annotations. If a comment cannot be linked to any method, such as if it refers to an import statement, it is discarded from the dataset due to not being useful for the scope of the study.

After linking comments to methods for each review round, a set of triplets m_s , m_r , and r_{nl} , where m_s and m_r represent the same abstracted method before and after the review round, and r_{nl} is a set of comments m_s received in this round. Here on out, all code components mentioned in any comment in r_{nl} are abstracted using the abstraction map obtained during the abstraction of m_s and m_r . For example, if the review comment mentions "change the type of sum to double" and that the variable sum has been abstracted to "VAR 1", the comment is transformed into "change the type of VAR 1 to double". Moreover, any camel case identifier that is not matched in the abstraction map but is present in the comment is replaced by the special token CODE. This ensures consistency in the representation of the code and comment that will be provided as input to the model and the representation of similar comments referring to different CODE elements.

(iii) **Filtering Out Noisy Comments**

Some code review comments present were unlikely to result in actual code changes, thus irrelevant to the study. For example, if two reviewers commented on the same method, one saying, "Looks good to me," and the other one asking for a change, "Please make this method static," any revised version of the method submitted afterward by the contributor would be the result of implementing the second comment rather than

the first one. So, to minimize the noise from such comments, the study devised an approach to automatically classify comments as relevant (likely to lead to code changes) or irrelevant. A dataset of 1,875 comments was created manually labeled as relevant or irrelevant, with 1,676 labeled as relevant and 199 as irrelevant. A set of keyword-based heuristics was then defined to filter out irrelevant comments. These heuristics aimed to remove the following types of comments:

- Useless one-word comments (e.g., "nice")
- Requests to change formatting with no impact on code (e.g., "please fix indentation")
- Thank you/approval messages (e.g., "looks good to me")
- Requests to add test code, which will not result in changes to the code under review (e.g., "please add tests for this method")
- Requests for clarification (e.g., "please explain")
- References to a previous comment that cannot be identified (e.g., "same as before")
- Requests to add comments, which were ignored in their study (e.g., "add to Javadoc")

By applying these heuristics to the complete triplet dataset, comments in r_{nl} that are unlikely to have triggered the code change that transformed m_s to m_r are filtered out from the triplets to create the paired dataset.

The dataset used in this study is collected from the replication package of the benchmark model and has already undergone the aforementioned preprocessing. However, additional preprocessing was needed to be performed. Features and targets for the training, validating, and testing datasets had to be merged. After merging, the data frames are converted to JSON format and uploaded to Hugging Face [4]. The dataset is loaded from Hugging Face datasets [5] and structured into a prompt format before undergoing the tokenization process. The data is structured into different sections: General System Instruction, Input, Context, and Response. The structuring of the data is based on the requirements of Code-Llama. It looks as follows:

"""

You are a powerful code-reviewing model. Your job is to improve Java code. You are given a Java method code as input.

You must output the improved version of the code provided as input.

Input:

{data_point["OriginalCode"]}

Response:

{data_point["ReviewedCode"]}

"""

The training, validating, and testing data size is as follows:

- Training data: 13,927 data points
- Evaluating data: 1,547 data points
- Testing data: 1,719 data points

(B) Model Selection

Among the potential candidates for pre-trained large language models, Code-Llama [6] is at the top. **Code-Llama** is a family of large language models for code based on Llama 2 [19], providing state-of-the-art performance among open models, infilling capabilities, support for large input contexts, and zero-shot instruction following ability for programming tasks. It is a code-specialized version of Llama 2 that was created by further training Llama 2 on its code-specific datasets, sampling more data from that same dataset for longer. Essentially, Code Llama features enhanced coding capabilities built on Llama-2. It can generate code and natural language about code from code and natural language prompts (e.g., "Write me a function that outputs the Fibonacci sequence.") It can also be used for code completion and debugging. It supports many popular programming languages, including Python, C++, Java, PHP, Typescript (Javascript), C#, and Bash. There are four release sizes of Code-Llama: 7B, 13B, 34B, and 70B parameters, respectively. Each model is trained with 500B tokens of code and code-related data, apart from 70B, which is trained on 1T tokens.

Llama 3 [28] is the latest LLM developed by Meta¹ with great improvements over its predecessor Llama 2. Currently, Llama 3 comes with 8B and 70B parameters, which demonstrate a wide range of industry benchmarks and offer new capabilities, including improved reasoning, code generation, and following instructions. Due to improvements in pre-training and post-training, the 8B and 70B parameter Llama 3 models are a major leap over Llama2 and establish a new state-of-the-art for LLMs at those scales. The training data for Llama 3 is over 15T tokens, seven times larger and four times the code than that used for Llama 2. For this study, we will be using the 8B parameter Llama 3 model. The natural and programming language pertaining to the size of this particular variant of Llama 3 makes it an ideal model to fine-tune and use for downstream tasks.

CodeT5+ [24] is a family of encoder-decoder LLMs for code that attempts to overcome the limitations code LLMs face regarding architectural and pertaining tasks. Through innovative pre-training tasks, which include span denoising and causal language modeling (CLM) tasks on code data and text-code contrastive learning, matching, and CLM tasks on text-code data, CodeT5+ can flexibly operate in encoder-only, decoder-only, and encoder-decoder modes to suit different downstream applications. These applications include text-to-code, code-to-text, code completion, and code retrieval tasks. CodeT5+ is available in 220M, 770M, 2B, 6B, and 16B parameters models. The first two employ the same architecture of T5², and the rest employ a "shallow encoder and deep decoder" architecture. These models are pre-trained on 51.5B tokens of a multilingual dataset consisting of Python, Java, Ruby, JavaScript, Go, PHP, C, C++, and C#. In this study the 220m and 770m parameter CodeT5+ models are selected.

CodeGemma [29] is a family of code-specialist LLM models by Google, based on the pre-trained 2B and 7B Gemma [30] checkpoints. CodeGemma are further trained on an additional 500 billion tokens of primarily English language data, mathematics, and code (C, C++, C#, Go, Java, JavaScript, Kotlin, Python, and Rust) to improve on logical and mathematical reasoning, and are suitable for code completion and generation. CodeGemma 2B was trained exclusively on Code Infilling and is meant for fast code completion and generation, especially in settings where latency and/or privacy are crucial. CodeGemma 7B training mix includes code infilling data (80%) and natural language. It can be used for code completion, as well as code and language understanding and generation. CodeGemma 7B Instruct was fine-tuned for instruction following on top of CodeGemma 7B. It's meant for conversational use, especially around code, programming, or mathematical reasoning topics. All the models have the same 8K token context size as their predecessors. In this study, the 7B parameter variant of CodeGemma is selected.

(C) Model Fine-tuning

Large language models pre-trained on massive datasets perform various tasks, including text generation, translation, summarization, and question-answering. The models selected for this study, in particular, are well-versed in natural and programming language-related tasks. As powerful as they are, they are not specifically tailored to a task. By leveraging the existing knowledge of pre-trained LLMs, we can further tailor them to a specific downstream task by fine-tuning them. Fine-tuning is the process by which pre-trained LLMs can be adapted to a more specialized task by further training the models on task-specific data while preserving their general knowledge from their previous training. By fine-tuning the selected models, they will learn from the domain-specific data (the dataset we create) to become more accurate and effective at generating documentation from source code.

As with pre-training the LLMs, fine-tuning is also a computationally demanding process. During the fine-tuning process, model parameters are adjusted to a specific domain. For a model to undergo full-supervised fine-tuning, all its trainable parameters will be adjusted in the process. This becomes computationally challenging when the size of the trainable parameters of LLMs to undergo the process is large. While CodeT5+ with 220M and 770M parameters can be fine-tuned without excessive computational demand, Code-Llama 7B, Llama 3 8B and CodeGemma 7B prove to be demanding. This demand results from graphics processing unit memory usage for loading the model, the gradients calculated during training, and the optimizer state that keeps track of the training variables. For example, for Code-Llama 7B, all of these take at least 112 GB of memory. To address

¹<https://about.meta.com/company-info/>

the computational expense and memory usage challenges associated with fine-tuning large models, parameter-efficient fine-tuning (PEFT) [7] strategies offer a practical and effective way to adapt these models to new tasks and domains.

PEFT is a technique designed to adapt LLMs to specific tasks without the need to update all of the model’s parameters. To fine-tune Code-Llama, Llama 3 and CodeGemma, a PEFT method called Low-Rank Adaptation of Large Language Models (LoRa) [8] will be used. LoRa freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. To further reduce the memory usage of the models during fine-tuning, we employ a technique to reduce the models’ size by modifying the precision of their weights. This procedure is called model quantization. The fundamental idea behind model quantization into lower precision is that neural networks don’t always need to use the entire range of a higher-order floating point to converge. QLoRa: Efficient Fine-Tuning of Quantized LLMs [9] is the technique used to quantize the Llama models. QLoRA backpropagates gradients through a frozen, 4-bit quantized pre-trained language model into Low-Rank Adapters (LoRA). By quantizing the models to an 8-bit precision, we can significantly reduce the memory usage required to fine-tune them. For example, for Code-Llama, the required memory is reduced to around 32 GB.

4 Results

The pre-trained language models chosen are fine-tuned on the pairs dataset and evaluated on the testing set. The fine-tuning process was successful as the models converged well. Figures 4, 5 and 6 show the validation loss that illustrates the convergence of the fine-tuning process for the pre-trained models. And Table 1 shows the training and validation losses for the models. These let us how well the models performed on the validation dataset, and will help provide a starting point on the performance difference we can expect from the models on the testing set.

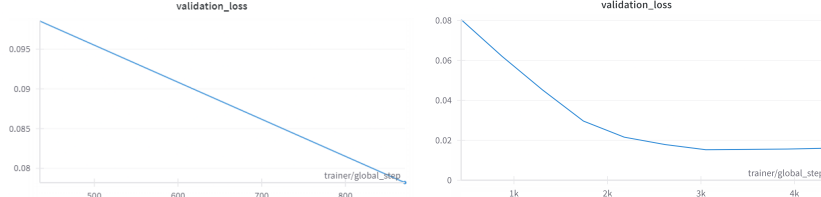


Figure 4: Validation Loss: CodeT5+ 220M on the left and CodeT5+ 770M on the right

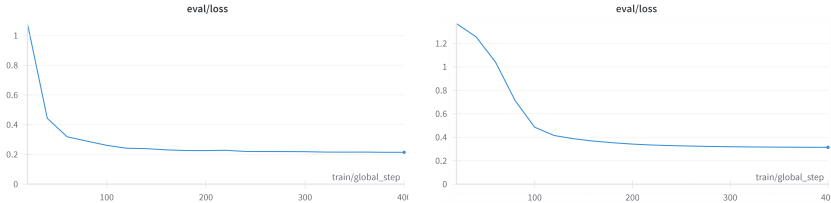


Figure 5: Validation Loss: Code-Llama 7B on the left and Llama-3 8B on the right

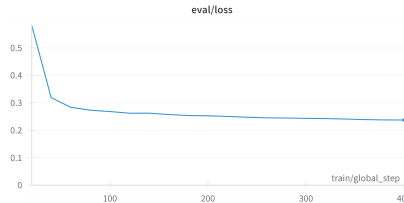


Figure 6: Validation Loss: CodeGemma 7B

Models	Training Loss	Validation Loss
Code-Llama 7B	0.2205	0.2139
Llama-3 8B	0.3241	0.3145
CodeT5+ 220M	0.082	0.0782
CodeT5+ 770M	0.000898	0.016037
CodeGemma 7B	0.2514	0.2375

Table 1: Training and Validation Loss

The mean, median, and standard deviation for the BLEU-4 and Levenshtein distance scores for all data points in the testing dataset have been calculated to better understand the change in the distribution of those measures and to have comparable values with the benchmark study.

Perfect Prediction		BLEU-4			Levenshtein Distance		
#	%	mean	median	st.dev.	mean	median	st.dev.
150	8.73%	0.7602	0.825	0.2288	0.2645	0.2595	0.17

Table 2: Code-Llama 7B Evaluation Scores

As per the illustration in Table 2, the model’s performance can be assessed with a reasonable degree of certainty from the evaluation metrics. The perfect prediction score shows that Code-Llama can accurately predict 8.73% of the testing data. This means the model generated an improved code implementation that is the same as the improved implementation provided by the contributor for 150 data points in the testing set. The high mean and median values, 0.7602 and 0.825, respectively, achieved in the BLEU-4 score indicate that the model’s overall prediction is closer to the reference code, and the low standard deviation value (0.2288) attests to that, as the BLEU-4 values for each data point have minimal disparity. The low mean and median values, 0.2645 and 0.2595, respectively, achieved by the Levenshtein distance score, show that the minimum edit required to transform the model-generated code to the reference code is rather small. From the 0.17 standard deviation value, we can generalize that the Levenshtein distance score disparity among the data points in the testing test is minimal.

Perfect Prediction		BLEU-4			Levenshtein Distance		
#	%	mean	median	st.dev.	mean	median	st.dev.
1	0.06%	0.4185	0.4058	0.1823	0.2143	0.1644	0.1656

Table 3: Llama-3 8B Evaluation Scores

Table 3 shows Llama-3 8B has a **0.06%** perfect prediction on the testing set, which equates to just one data-point. It has relatively low BLEU-4 score with a mean and median values of **0.4185** and **0.4058** respectively. The Levenshtein distance score for this model is very low, with even the lowest median value of **0.1644**. This value tells us that the change needed for the model’s generation to match the ground truth is very small. Table 4 illustrates the performance of CodeGemma 7B. CodeGemma has a **0.0%** perfect prediction on the test set and has low BLEU-4 value with a mean and median of **0.0166** and **0.0161** respectively. CodeGemma also exhibits a large Levenshtein distance score with mean and median values of **0.6723** and **0.653** respectively. This shows that the model’s prediction requires more change to turn it into the ground truth. Given its standard deviation value of 0.089, it can be inferred that the implication of the Levenshtien distance score is consistent accross the testing set.

The CodeT5+ models are shown to perform well on Table 5 and Table 6. The 220 million parameter CodeT5+ model has a perfect prediction of **27** which equates to **1.57%** of the testing dataset. This variant of the CodeT5+ model has a relatively lower BLEU-4 scores compared to Code-Llama 7B and OpenNMT-tf models, but has the lowest median Levenshtein distance score of **0.1644**. The 770

Perfect Prediction		BLEU-4			Levenshtein Distance		
#	%	mean	median	st.dev.	mean	median	st.dev.
0	0.0%	0.0166	0.0161	0.0306	0.6723	0.653	0.089

Table 4: CodeGemma 7B Evaluation Scores

Perfect Prediction		BLEU-4			Levenshtein Distance		
#	%	mean	median	st.dev.	mean	median	st.dev.
27	1.57%	0.4409	0.4286	0.1905	0.2079	0.1667	0.1711

Table 5: CodeT5+ 220M Evaluation Scores

million parameter CodeT5+ model makes a perfect prediction for **1.34%** of the testing set which amounts to **23** data-points. It has a slightly lower BLEU-4 and Levenshtein distance scores, but is in the hundredth decimal place. Compared to the 220M parameter model, it has lower standard deviation for both the BLEU-4 and Levenshtein distance scores, indicating the consistency of metric evaluation scores across the testing set.

5 Evaluation

Table 7 illustrates the perfect prediction, BLEU-4, and Levenshtein distance scores for Code-Llama 7B, Llama-3 8B, CodeT5+ 220M and 770M, CodeGemma 7B and OpenNMT-tf (1-encoder by 2-decoder) code predictions. Code-Llama perfectly predicted three times (**5.85% more**) the amount the benchmark model has for the testing data, showcasing its superior performance. The mean and median BLEU-4 scores for Code-Llama and OpenNMT-tf models are relatively similar, with only a difference in the second decimal point by 1. Despite that, we can see a slight increase in value for the OpenNMT-tf model. However, from the difference in the standard deviation values, it can be inferred that the BLEU-4 scores for each improved code implementation by the benchmark model have less disparity, meaning the model is more consistent across different test examples. Although the Levenshtein distance score difference between these two models is very close to one another, the overall improved code implementations from the benchmark model are shown to require fewer edits to match their respective reference code.

Despite having the lowest mean Levenshtein distance score, CodeT5+ 220M comes third in line for the highest number of perfect prediction. The CodeT5+ 700M parameters variant surprisingly underperforms in perfect prediction, BLEU-4 and Levenshtein distance scores compared to its 220M parameter variant model. It achieves a perfect prediction for **23** (1.34%) of the testing data, a difference in four test cases compared to its fewer parameter size variant. Interestingly, we only see a slight difference in the BLEU-4 and Levenshtein distances scores, where there is a difference in value is in the hundredth decimal place. The fourth better performing model is the Llama-3 8B with only one perfect prediction (0.06%) of the testing data. Similarly to the CodeT5+ models, the Llama-3 model exhibits a small change in the BLEU-4 and Levenshtein distance score, but nonetheless has the lowest median score for the latter. CodeGemma 7B, on the other hand, is the least performing model showing the lowest values for perfect prediction (0.0% of the testing set) and BLEU-4 and the highest for Levenshtein distance score. Its low value in standard deviation for both the BLEU-4 and Levenshtein distance score indicate that the model’s prediction is not consistent with respect to the evaluation metrics applied.

Perfect Prediction		BLEU-4			Levenshtein Distance		
#	%	mean	median	st.dev.	mean	median	st.dev.
23	1.34%	0.4356	0.4262	0.1873	0.2126	0.1733	0.1674

Table 6: CodeT5+ 770M Evaluation Scores

Perfect Prediction		BLEU-4			Levenshtein Distance		
#	%	mean	median	st.dev.	mean	median	st.dev.
OpenNMT-tf (1-encoder by 2-decoder)							
50	2.91%	0.7706	0.8315	0.1929	0.2383	0.2000	0.1670
Code-Llama 7B							
150	8.73%	0.7602	0.825	0.2288	0.2645	0.2595	0.17
CodeT5+ 220m							
27	1.57%	0.4409	0.4286	0.1905	0.2079	0.1667	0.1711
CodeT5+ 770m							
23	1.34%	0.4356	0.4262	0.1873	0.2126	0.1733	0.1674
Llama-3 8B							
1	0.06%	0.4185	0.4058	0.1823	0.2143	0.1644	0.1656
Code-Gemma 7B							
0	0.0%	0.0166	0.0161	0.0306	0.6723	0.653	0.089

Table 7: Evaluation Scores for Code-Llama 7B, CodeT5+ 220m, CodeT5+ 770m, Llama-3 8B, Code-Gemma 7B and OpenNMT-tf

Code-Llama’s superior performance can be attributed to the rich representations and patterns learned from large, diverse datasets during the pertaining phase. This gives it a better understanding of English and programming languages. Since its weights are already initialized with meaningful values from pertaining, the fine-tuning process can converge faster, requiring fewer training iterations to achieve good performance on the target task. The model is also fairly large, with 7 billion parameters, which gives it an advantage in generalizing knowledge. This especially becomes important when fine-tuning it because it can leverage the knowledge in its large parameter space. Although the CodeT5+ model variants chosen are much smaller in size, all the pre-trained models chosen for this study have a rich knowledge in their large parameter space they can draw from to perform well at the downstream task at hand.

Although it’s expected for the models to perform well, the results from the metrics chosen show it to be otherwise. This is a very important observation to make, as models that are better are shown to be less performing. For example, Llama-3 8B is a larger model compared to Code-Llama 7B and is pre-trained on a larger dataset. It has also been shown to outperform the 7 billion parameter variant of Code-Llama on various downstream tasks, it still performs less on a code review task. The results provide interesting observation as the smaller models of CodeT5+ perform better than the larger models: Llama-3 8B and CodeGemma 7B. From this observation we can infer that perfect prediction may not be a metric we can primarily rely on. It is also important to perform qualitative analysis of each model’s predictions to assess the validity of the different interpretation of improved code representation adopted by the models. The benchmark study performed a qualitative analysis of sample codes generated by the model by assembling a team of professionals. This aims to categorize code changes implemented by the model and to understand the meaningfulness of non-perfect predictions, providing more insights from the scores measured by the evaluation metrics. This isn’t performed in this study.

6 THREATS TO VALIDITY

The study can be strengthened by increasing the number of models for both sides under comparison to get a more comprehensive and conclusive result. Performing a qualitative analysis of the code

generated by the models to ensure its relevance can also improve the process of assessing model performance. This is important for two reasons: (1) analyzing perfectly predicted codes from each model under comparison helps identify the code complexity the models can better learn and predict. Doing this for each model's prediction is also important because the code complexity each model can learn better can be different. (2) The second reason for performing qualitative analysis is that the code not perfectly predicted by the models isn't necessarily faulty. Just because it's not perfectly predicted it doesn't mean it's not a valid alternate implementation. The lack of qualitative analysis restricts a more refined interpretation of the BLEU-4 and Levenshtein distance evaluation scores. Moreover, the results from BLEU-4 and Levenshtein distance are influenced by the syntactic sugar of source code and the input code provided to the model, which is usually quite similar to the one to be produced as output. The small dataset size also restricts the model's ability to learn rich representations and limits its knowledge scope. It's also important to note that quantizing Code-Llama, Llama-3 and CodeGemma to a lower precision can lead to a drop in the models' performance.

7 CONCLUSION

This comparative study evaluates whether using pre-trained LLMs or transformer-based models trained from scratch to automate the code review process is more advantageous. The research focused on transforming Java methods from their original submissions (m_s) to their revised versions (m_r). Although the results from the BLEU-4 and Levenshtein distance scores are comparably similar, the study's results indicate that Code-Llama, with its 7 billion parameters, outperforms the vanilla transformer-based model (OpenNMT-tf) in terms of perfect prediction. To address the study's limitations and further enhance its results, future work should consider expanding the model comparison (both pre-trained and vanilla models), performing qualitative analysis, enriching the dataset, and involving human reviewers in the evaluation process. By pursuing these avenues, future research can build upon the current study's findings to develop more robust, efficient, and user-friendly tools for automating the code review process, ultimately contributing to the advancement of software development practices.

REFERENCES

- [1] Palvannan, Nivishree, and Chris Brown. "Suggestion Bot: Analyzing the Impact of Automated Suggested Changes on Code Reviews." arXiv preprint arXiv:2305.06328 (2023).
- [2] Tufano, Rosalia, et al. "Towards automating code review activities." 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). IEEE, 2021.
- [3] Lu, Junyi, et al. "LLaMA-Reviewer: Advancing Code Review Automation with Large Language Models through Parameter-Efficient Fine-Tuning." 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). IEEE, 2023.
- [4] <https://huggingface.co/>
- [5] https://huggingface.co/datasets/YabTad/CodeRev_BiData/tree/main
- [6] Roziere, Baptiste, et al. "Code llama: Open foundation models for code." arXiv preprint arXiv:2308.12950 (2023).
- [7] Xu, Lingling, et al. "Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment." arXiv preprint arXiv:2312.12148 (2023).
- [8] Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." arXiv preprint arXiv:2106.09685 (2021).
- [9] Dettmers, Tim, et al. "Qlora: Efficient finetuning of quantized llms." Advances in Neural Information Processing Systems 36 (2024).
- [10] G. Klein, Y. Kim, Y. Deng, V. Nguyen, J. Senellart, and A. M. Rush, "Opennmt: Neural machine translation toolkit," arXiv preprint arXiv:1805.11462, 2018.
- [11] "Opennmt-tf. <https://github.com/OpenNMT/OpenNMT-tf/>."
- [12] "Github. <https://github.com/>."
- [13] "Gerrit. <https://www.gerritcodereview.com> (last access: 11/08/2018)."

- [14] M. Tufano, J. Pantiuchina, C. Watson, G. Bavota, and D. Poshyvanyk, "On learning meaningful code changes via neural machine translation," in *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019, 2019*, pp. 25–36.
- [15] M. Tufano, C. Watson, G. Bavota, M. D. Penta, M. White, and D. Poshyvanyk, "An empirical study on learning bug-fixing patches in the wild via neural machine translation," *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 4, pp. 19:1–19:29, 2019.
- [16] C. Watson, M. Tufano, K. Moran, G. Bavota, and D. Poshyvanyk, "On learning meaningful assert statements for unit test cases," in *Proceedings of the 42nd International Conference on Software Engineering, ICSE 2020, 2020*, p. To Appear.
- [17] "Lizard. <https://github.com/terryyin/lizard/>."
- [18] "src2abs. <https://github.com/micheletufano/src2abs/>."
- [19] Touvron, Hugo, et al. "Llama 2: Open foundation and fine-tuned chat models." *arXiv preprint arXiv:2307.09288* (2023).
- [20] Papineni, Kishore, et al. "Bleu: a method for automatic evaluation of machine translation." *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 2002.
- [21] Levenshtein, Vladimir I. "Binary codes capable of correcting deletions, insertions, and reversals." *Soviet physics doklady*. Vol. 10. No. 8. 1966.
- [22] Sghaier, Oussama Ben, and Houari Sahraoui. "Improving the Learning of Code Review Successive Tasks with Cross-Task Knowledge Distillation." *arXiv preprint arXiv:2402.02063* (2024).
- [23] Tufano, Rosalia, et al. "Using pre-trained models to boost code review automation." *Proceedings of the 44th international conference on software engineering*. 2022.
- [24] Wang, Yue, et al. "Codet5+: Open code large language models for code understanding and generation." *arXiv preprint arXiv:2305.07922* (2023).
- [25] Li, Raymond, et al. "Starcode: may the source be with you!." *arXiv preprint arXiv:2305.06161* (2023).
- [26] Liu, Kaibo, et al. "LLM-Powered Test Case Generation for Detecting Tricky Bugs." *arXiv preprint arXiv:2404.10304* (2024).
- [27] Zhang, Quanjun, et al. "A Systematic Literature Review on Large Language Models for Automated Program Repair." *arXiv preprint arXiv:2405.01466* (2024).
- [28] <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>
- [29] Team, CodeGemma. "Codegemma: Open code models based on gemma." *arXiv preprint arXiv:2406.11409* (2024).
- [30] Team, Gemma, et al. "Gemma: Open models based on gemini research and technology." *arXiv preprint arXiv:2403.08295* (2024).